

Debugging Techniques



Introduction

- Kernel programming brings its own, unique debugging challenges.
- Kernel code cannot be easily executed under a debugger, nor can it be easily traced, because it is a set of functionalities not related to a specific process.
- Kernel code errors are extremely hard to reproduce.



Kernel- vs User-Space Debugging

- Difficulty is higher
 - No built-in debuggers
 - Bugs may be hard to reproduce
- Stakes are higher
 - Fault in kernel can bring down whole system or cause unexplained behaviors



Types of Bugs

- Incorrect code
 - Example: not storing correct value in proper place
- Synchronization error
 - Example: not properly locking a shared variable
- Incorrectly managing hardware
 - Example: sending wrong operation to wrong control register



Pitfalls

- Beware NULL or garbage pointers
- Zero-out memory before using
- Do not re-create the wheel
 - Use functions already available (e.g. linked list, strings)
- Beware of any warnings in compilation
- Minimize complexity



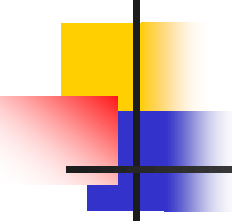
Debugging Support in the Kernel

- Under the “kernel hacking” menu
 - Not supported by all architectures
- **CONFIG_DEBUG_KERNEL**
 - Enables other debugging features
- **CONFIG_DEBUG_SLUB**
 - Checks kernel memory allocation functions
 - Memory overrun
 - Memory initialization



Debugging Support in the Kernel

- **CONFIG_LOCKUP_DETECTOR**
 - Detect hard and soft lockups
 - Softlockups – cause kernel to loop for more than 60 seconds
 - Hardlockups – cause cpu (or core) to loop for more than 60 seconds



Debugging Support in the Kernel

- **CONFIG_DEBUG_PAGEALLOC**
 - Full Pages are removed from the kernel address space when freed
- **CONFIG_DEBUG_SPINLOCK/_SLEEP**
 - Catches operations on uninitialized spinlocks and double unlocking
- **CONFIG_DEBUG_MUTEXES**
 - Detects and reports various mutex violations



Debugging Support in the Kernel

- **CONFIG_DEBUG_INFO**
 - Enables `gdb` debugging
- **CONFIG_DEBUG_ATOMIC_SLEEP**
 - Reporting if calling a routine that may sleep inside a critical section
- **CONFIG_KGDB***
 - Remotely debug the kernel using `gdb`



Debugging Support in the Kernel

- **CONFIG_MAGIC_SYSRQ**
 - For debugging system hangs
- **CONFIG_DEBUG_STACKOVERFLOW**
 - Helps track down kernel stack overflows
- **CONFIG_DEBUG_STACK_USAGE**
 - Monitors stack usage and makes statistics available via magic SysRq key



Debugging Support in the Kernel

- **CONFIG_KALLSYMS**
 - Causes kernel symbol information to be built into the kernel. Enabled by default.
- **CONFIG_FRAME_POINTER**
 - Produces more reliable stack backtraces
- **CONFIG_PROFILING**
 - For performance tuning



printk (vs. printf)

- Lets one classify messages according to their priority by associating with different loglevels, or priorities, with the messages.
 - `printk(KERN_DEBUG "Here I am: %s:%i\n", __FILE__, __LINE__);`
- Eight possible loglevels (0 - 7), defined in `<linux/kern_levels.h>`



printk (vs. printf)

- **KERN_EMERG**
 - For emergency messages
- **KERN_ALERT**
 - For a situation requiring immediate action
- **KERN_CRIT**
 - Critical conditions, related to serious hardware or software failures



printk (vs. printf)

- **KERN_ERR**

- Used to report error conditions; device drivers often use it to report hardware difficulties

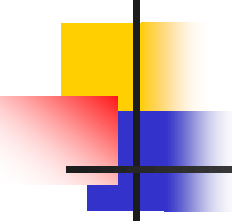
- **KERN_WARNING**

- Warnings for less serious problems



printk (vs. printf)

- **KERN_NOTICE**
 - Normal situations worthy of note (e.g., security-related)
- **KERN_INFO**
 - Informational messages
- **KERN_DEBUG**
 - Used for debugging messages



printk (vs. printf)

- Without specified priority
 - `DEFAULT_MESSAGE_LOGLEVEL = KERNEL_WARNING`
- If current priority < `console_loglevel`
 - `console_loglevel` initialized to `DEFAULT_CONSOLE_LOGLEVEL`
 - Message is printed to the console one line at a time



printk (vs. printf)

- If both `klogd` and `syslogd` are running
 - Messages are appended to `/var/log/messages`
- `klog` daemon doesn't save consecutive identical lines, only the first line + the number of repetitions



printk (vs. printf)

- `console_loglevel` can be modified using `/proc/sys/kernel/printk`
 - Contains 4 values
 - Current loglevel
 - Default log level
 - Minimum allowed loglevel
 - Boot-timed default loglevel
 - `echo 6 > /proc/sys/kernel/printk`



How Messages Get Logged

- `printk` writes messages into a circular buffer that is `__LOG_BUF_LEN` bytes
 - If the buffer fills up, `printk` wraps around and overwrite the beginning of the buffer
 - Can specify the `-f <file>` option to `klogd` to save messages to a specific file



How Messages Get Logged

- Reading from `/proc/kmsg` consumes data
- `syslog` system call can leave data for other processes (try `dmesg` command)



Rate Limiting

- Too many messages may overwhelm the console
- To reduce repeated messages, use
 - `int printk_ratelimit(void);`
- Example

```
if (printk_ratelimit()) {  
    printk(KERN_NOTICE "The printer is still on fire\n");  
}
```



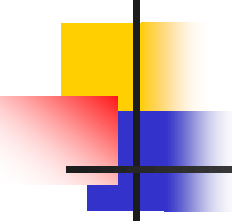
Rate Limiting

- To modify the behavior of **`printk_ratelimit`**
 - `/proc/sys/kernel/printk_ratelimit`
 - Number of seconds before re-enabling messages
 - `/proc/sys/kernel/printk_ratelimit_burst`
 - Number of messages accepted before rate limiting



printk from userspace

- Put messages in the printk buffer
- Example usage:
 - `echo "Hello Kernel-World" > /dev/kmsg`
- Useful to determine ordering between userspace actions and kernel actions



Using the `/proc` Filesystem

- Exports kernel information
- Each file under `/proc` tied to a kernel function
 - `/proc/cpuinfo`, `/proc/meminfo`
- Will give in-depth example after introducing character driver next week



The `ioctl` Method

- Implement additional commands to return debugging information
 - Advantages
 - More efficient
 - Does not need to split data into pages
 - Can be left in the driver unnoticed



Debugging by Watching

- **strace** command
 - Shows system calls, arguments, and return values
 - No need to compile a program with the `-g` option
 - `-t` to display when each call is executed
 - `-T` to display the time spent in the call
 - `-e` to limit the types of calls
 - `-o` to redirect the output to a file



Debugging System Faults

- A fault usually ends the current process, while the system continues to work
- Potential side effects
 - Hardware left in an unusable state
 - Kernel resources in an inconsistent state
 - Corrupted memory
- Common remedy
 - Reboot



OOPS Message

- State of the system when an error occurred
- Useful for debugging
- May or may not be useful



Example OOPS

```
static int hello_init(void)
{
    printk(KERN_ALERT "Hello, world\n");
    *(int *)0 = 0;
    return 0;
}
```

```
Hello, world
BUG: unable to handle kernel NULL pointer dereference at (null)
IP: [<fffffffffffa000f012>] hello_init+0x12/0x21 [hello]
PGD 32e006067 PUD 32cfaa067 PMD 0
Oops: 0002 [#1] PREEMPT SMP
Modules linked in: hello(O+) fuse nouveau [last unloaded: hello]
CPU: 0 PID: 8040 Comm: insmod Tainted: G O 3.13.7 #4
Hardware name: System manufacturer System Product Name/P6T6 WS REVOLUTION, BIOS 0507 7/02/2009
task: ffff8800ba86c350 ti: ffff88030001a000 task.ti: ffff88030001a000
RIP: 0010:[<fffffffffffa000f012>] [<fffffffffffa000f012>] hello_init+0x12/0x21 [hello]
RSP: 0018:ffff88030001bd68 EFLAGS: 00010292
RAX: 000000000000000c RBX: fffffffffa000f000 RCX: 0000000000000000
RDX: 0000000000000001 RSI: ffff88033fc0cf48 RDI: 00000000ffffffff
RBP: ffff88030001bd68 R08: 0000000000000400 R09: ffffffff8173da24
R10: ffffffff8173da24 R11: 000000000000b8ac R12: 0000000000000000
R13: 0000000000000000 R14: ffff88030001bef8 R15: 0000000000000001
FS: 00007f05d0d48700(0000) GS:ffff88033fc00000(0000) knlGS:0000000000000000
CS: 0010 DS: 0000 ES: 0000 CR0: 0000000080050033
CR2: 0000000000000000 CR3: 000000032ff6b000 CR4: 00000000000007f0
Stack:
 ffff88030001bdd8 ffffffff81000290 0000000000000000 ffff88030001bef8
 ffff88030001bdc8 ffffffff8104e542 0000000000000000 00000000ffffffff
 fffffffffa000f090 0000000000000001 fffffffffa000f090 0000000000000001
Call Trace:
 [<ffffffffff81000290>] do_one_initcall+0x7f/0x107
 [<ffffffffff8104e542>] ? __blocking_notifier_call_chain+0x4c/0x5a
 [<ffffffffff8107f900>] load_module+0x1166/0x13e1
 [<ffffffffff8107d02e>] ? mod_kobject_put+0x45/0x45
 [<ffffffffff8107fc7c>] Sys_finit_module+0x56/0x6c
 [<ffffffffff8133cd19>] tracesys+0xd0/0xd5
Code: <c7> 04 25 00 00 00 00 00 00 00 00 31 c0 5d c3 55 48 c7 c7 6c f0 00
RIP [<fffffffffffa000f012>] hello_init+0x12/0x21 [hello]
RSP <ffff88030001bd68>
CR2: 0000000000000000
---[ end trace 90412cd9054bc448 ]---
```

Hello, world

BUG: unable to handle kernel NULL pointer dereference at

IP: [<fffffffffa000f012>] hello_init+0x12/0x21 [hello]

PGD 32e006067 PUD 32cfaa007 PMD 0

Oops: 0002 [#1] PREEMPT SMP

Modules linked in: hello(O+) fu

CPU: 0 PID: 8040 Comm: insmod T

Hardware name: System manufactu

task: ffff8800ba86c350 ti: ffff

RIP: 0010:[<fffffffffa000f012>]

RSP: 0018:ffff88030001bd68 EFLAGS: 00010292

RAX: 000000000000000c RBX: fffffffffa000f000 RCX: 0000000000000000

RDX: 0000000000000001 RSI: ffff88033fc0cf48 RDI: 00000000ffffffff

RBP: ffff88030001bd68 R08: 0000000000000400 R09: ffffffff8173da24

R10: ffffffff8173da24 R11: 000000000000b8ac R12: 0000000000000000

R13: 0000000000000000 R14: ffff88030001bef8 R15: 0000000000000001

FS: 00007f05d0d48700(0000) GS:ffff88033fc00000(0000) knlGS:0000000000000000

CS: 0010 DS: 0000 ES: 0000 CR0: 0000000080050033

CR2: 0000000000000000 CR3: 000000032ff6b000 CR4: 00000000000007f0

Stack:

ffff88030001bdd8 ffffffff81000290 0000000000000000 ffff88030001bef8

ffff88030001bdc8 ffffffff8104e542 0000000000000000 00000000ffffffff

fffffffffa000f090 0000000000000001 fffffffffa000f090 0000000000000001

Call Trace:

[<ffffffff81000290>] do_one_initcall+0x7f/0x107

[<ffffffff8104e542>] ? __blocking_notifier_call_chain+0x4c/0x5a

[<ffffffff8107f900>] load_module+0x1166/0x13e1

[<ffffffff8107d02e>] ? mod_kobject_put+0x45/0x45

[<ffffffff8107fc7c>] Sys_finit_module+0x56/0x6c

[<ffffffff8133cd19>] tracesys+0xd0/0xd5

Code: <c7> 04 25 00 00 00 00 00 00 00 00 31 c0 5d c3 55 48 c7 c7 6c f0 00

RIP [<fffffffffa000f012>] hello_init+0x12/0x21 [hello]

RSP <ffff88030001bd68>

CR2: 0000000000000000

---[end trace 90412cd9054bc448]---

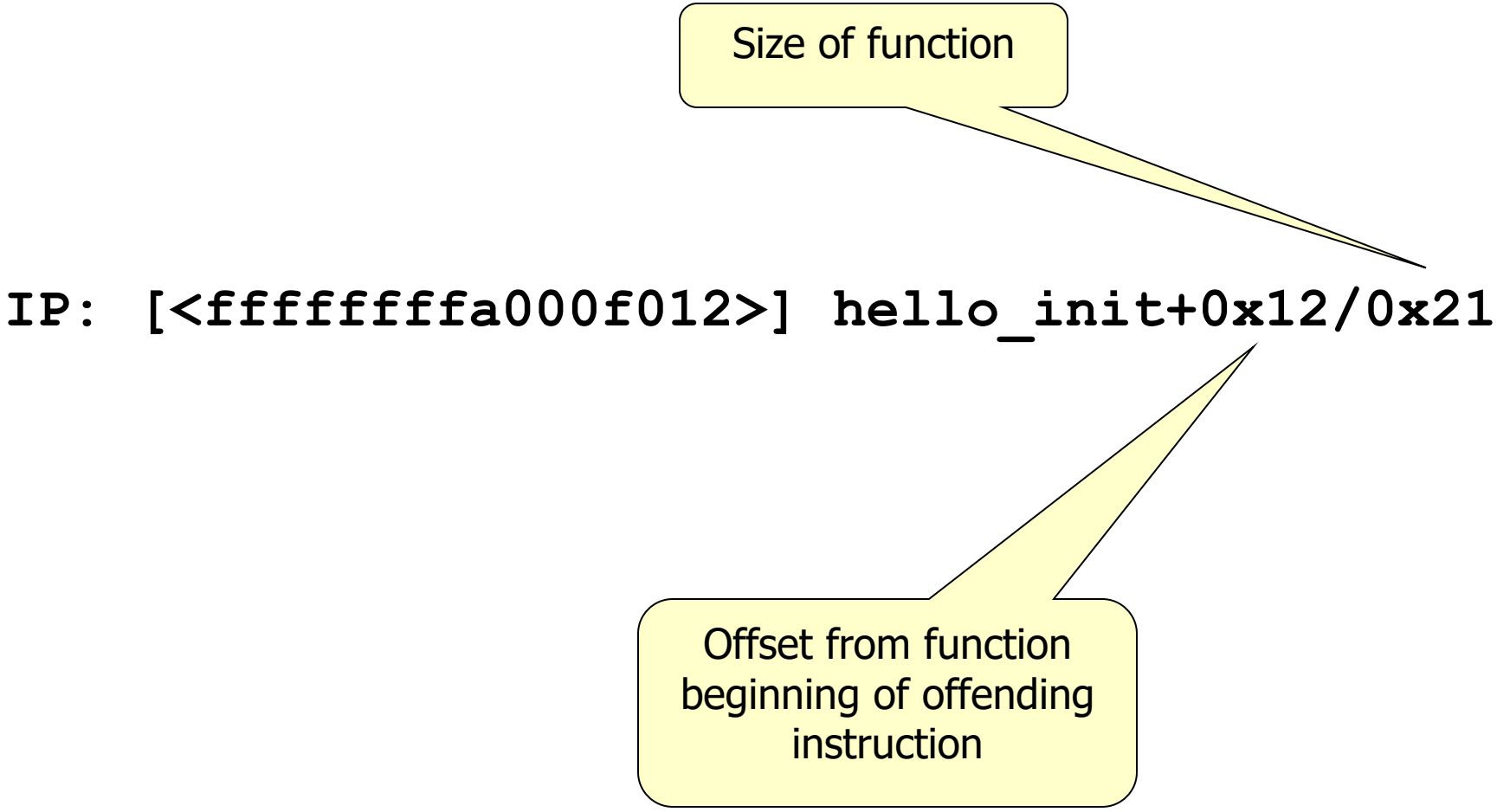
(null)

Error
message

Instruction Pointer
When Error Occurred
(Function)

Call Trace

Size of function



The diagram illustrates a call instruction in assembly. The instruction is `IP: [<fffffffffa000f012>] hello_init+0x12/0x21`. A yellow callout box labeled 'Size of function' points to the `0x21` part of the instruction. Another yellow callout box labeled 'Offset from function beginning of offending instruction' points to the `0x12` part of the instruction.

IP: [<fffffffffa000f012>] hello_init+0x12/0x21

Offset from function
beginning of offending
instruction


```
$ gdb hello.ko
```

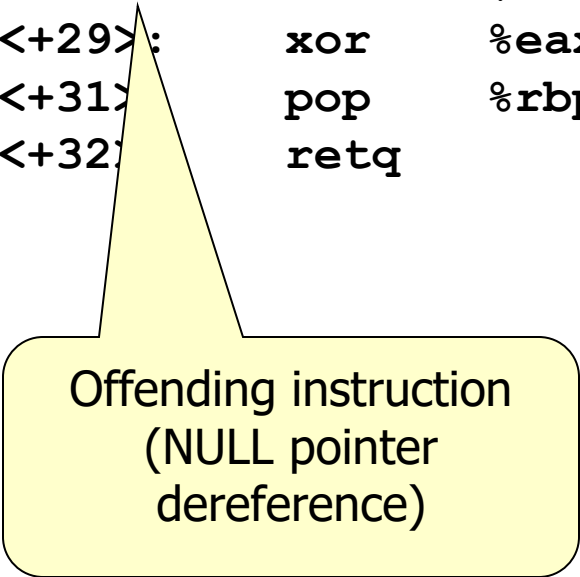
```
Reading symbols from /home/mark/tmp_module/hello.ko...done.
```

```
(gdb) disassemble hello_init
```

```
Dump of assembler code for function hello_init:
```

```
0x0000000000000024 <+0>: push    %rbp
0x0000000000000025 <+1>: mov     $0x0,%rdi
0x000000000000002c <+8>: xor     %eax,%eax
0x000000000000002e <+10>: mov     %rsp,%rbp
0x0000000000000031 <+13>: callq   0x36 <hello_init+18>
0x0000000000000036 <+18>: movl    $0x0,0x0
0x0000000000000041 <+29>: xor     %eax,%eax
0x0000000000000043 <+31>: pop     %rbp
0x0000000000000044 <+32>: retq
```

```
End of assembler dump.
```



Offending instruction
(NULL pointer
dereference)

0x24 + 0x12
func offset
start

```
(gdb) list *0x36
0x36 is in hello_init
(/home/mark/tmp_module/hello.c:8).
3  MODULE_LICENSE("Dual BSD/GPL");
4
5  static int hello_init(void)
6  {
7      printk(KERN_ALERT "Hello, world\n");
8      *(int *)0 = 0;
9      return 0;
10 }
11
12 static void hello_exit(void)
(gdb)
```



Oops Messages

- Require `CONFIG_KALLSYMS` option turned on to see meaningful messages
- Other tricks
 - `0xa5a5a5a5` on stack → memory not initialized



Asserting Bugs and Dumping Information

- **BUG()** and **BUG_ON(conditional)**
 - Cause an oops, which results in a stack trace and an error message
- **panic()**
 - Causes an oops and halts the kernel

```
if (terrible_thing)
    panic("terrible_thing is %ld!\n",
        terrible_thing);
```



Asserting Bugs and Dumping Information

- `dump_stack()`
 - Dumps contents of the registers and a function backtrace to the console without an oops



System Hangs

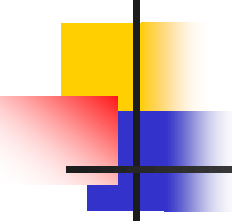
- Keyboard lockups, but other things are still working
 - Use the “magic SysRq key”
 - To enable magic SysRq
 - Compile kernel with `CONFIG_MAGIC_SYSRQ` on
 - `echo 1 > /proc/sys/kernel/sysrq`
 - To trigger magic SysRq
 - `Alt-SysRq-<command>`
 - `echo <command> > /proc/sysrq-trigger`



System Hangs

- Key

- **k**: “Secure attention key”, kills all processes running on the current console with clean terminal.
- **s**: emergency synchronization of all disks
- **u**: Command **s** + **u**mount and remount all disks in read-only mode. Saving a lot of filesystem.
- **b**: reboot, make sure to synchronize and remount the disks first



System Hangs

- **p**: prints processor registers information
- **t**: prints the current task list
- **m**: prints memory information
- **r**: Turns of the keyboard raw mode; crashed application may left the keyboard in a strange state.
- See `sysrq.txt` for more
- Precaution for chasing system hangs
 - Mount all disks as read-only



System Hangs

unRaw (take control of keyboard back from X),
tErminate (send SIGTERM to all processes, allowing them to terminate gracefully),
kIll (send SIGKILL to all processes, forcing them to terminate immediately),
Sync (flush data to disk),
Unmount (remount all filesystems read-only),
reBoot.

"Reboot Even If System Utterly Broken"



Debugging by Querying

- Techniques are available to driver developers for querying the system:
 - Creating a file in the /proc filesystem.
 - Using the ioctl driver method
 - Exporting attributes via sysfs.



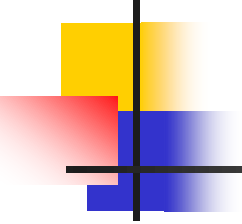
Using the /proc Filesystem

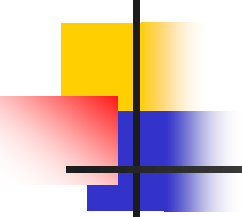
- Exports kernel information
- File under /proc is tied to a kernel function that generates the file's "content" when the file is read.
- /proc/modules
- /proc filesystem is dynamic and read only files.
- Adding files under /proc is discouraged.



Using ioctl Method

- Is a system call that acts as a file descriptor, it receives a number that identifies a command to be performed and another argument, usually a pointer.
- Relevant data structures are copied from the driver to user space.

- 
-
- Information fetching is somewhat difficult than using `/proc`, as it needs another program to issue the `ioctl` and display the result.
 - Information-retrieval command is left in the driver when debugging.



```
int main(int argc, char **argv) {
    char bytes[2] = {11, 0};
    if(argc==2) bytes[1] = atoi(argv[1]);
    else { fprintf(stderr, "%s: need a single arg\n",
argv[0]); exit(1); }
    if(ioctl(STDIN_FILENO, TIOCLINUX, bytes)<0)
    {
        fprintf(stderr, "%s: ioctl(stdin, TIOCLINUX) :
%s\n", argv[0], strerror(errno));
        exit(1); } exit(0); }
```